

mapler

 R-CMD-check.yaml **passing**

The goal of mapler is to provide functions to investigate how maple syrup production could be impacted by climate change.

Installation

You can install the development version of mapler from [GitHub](#) with:

```
# install.packages("pak")
pak::pak("matthewwhite1/mapler")
```

Downloading Climate Data

This package provides three functions for downloading climate data: `download_ghcnd()`, `download_loca2()`, and `download_prism()`. These functions are useful for downloading both historic climate data and future climate projections.

Once data is downloaded, it can be loaded into R using `loca_t_rast()` or `prism_t_rast()`, “t” meaning temperature. These functions take filepaths of where the data is downloaded and return the tmin and tmax raster stacks.

Calculating Sap Days

Maple sap tapping is highly dependent on the freeze/thaw cycle. This package calls a “sap day” a day where the temperature switched between freezing and thawing during the day.

The function `sap_day()` takes in tmin and tmax raster stacks and aggregates them into a yearly sap day stack. This combination of tmin and tmax is used to calculate the proportion of sap days in a year at every pixel. This function can be used for future climate projection with LOCA2 data.

The `sap_day()` function can be used with LOCA2 data like so:

```
# Load in rasters
loca_rast <- loca_t_rast("F:/Data/LOCA2/ACCESS-CM2/0p0625deg/r1i1p1f1/")

# Calculate yearly sap days (LOCA2 temps are in Kelvin)
k_upper <- 2.2 + 273.15
k_lower <- -1.1 + 273.15
loca_sap_day <- sap_day(loca_rast$tmax, loca_rast$tmin,
                        t_upper = k_upper, t_lower = k_lower)

# Shift raster to correct longitude to match WGS84
test_loca <- loca_sap_day$proportion |>
  terra::shift(dx = -360)
```

Package Data

This package contains a few datasets from various sources. The `farms` datasets (`us_farms`, `canada_farms`, `quebec_farms`) contain maple farm names and addresses scraped from the internet. The `farms_coords` dataset contains geocoded latitude and longitude coordinates of these farms.

This package also contains two datasets to assist in downloading climate data:

`loca2_model_names` and `ghcnd_stations`.

For an example of what `sap_day()` can do, this package contains an example raster called `test_loca_sap_day`. This was created by downloading historical and scenario ssp585 tmin and tmax data from the LOCA2 model ACCESS-CM2, loading the rasters into R using `loca_t_rast()`, calculating yearly sap day proportions using `sap_day()`, shifting the longitude by -360 to match WGS84, and aggregating by a factor of 20 to decrease the file size. It can be accessed like so:

```
test_loca_file <- system.file("extdata", "test_loca_sap_day.tif",
                             package = "mapler")
sap_prop <- terra::rast(test_loca_file)
```

Sen's Slope

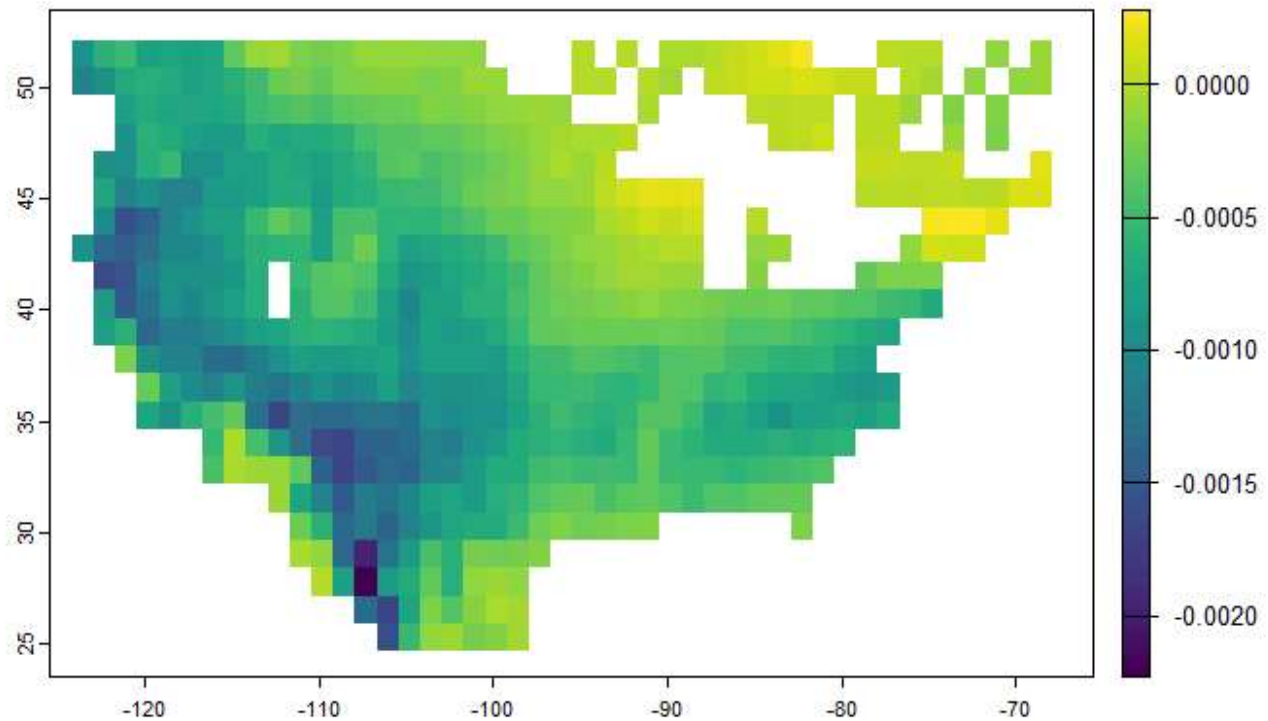
This package contains a function called `sens_slope_rast()` that calculates the Sen's slope for every pixel in a raster stack. This function is based on the Rcpp-optimized `sens_slope()`

function, which can be used in contexts separate from maple syrup. These functions can be used like so:

```
library(mapler)
library(terra)
#> Warning: package 'terra' was built under R version 4.4.3
#> terra 1.8.60

# Get Sen's slope of vector of random values
x <- rnorm(1000)
sens_slope(x)
#> $estimates
#> Sen's slope
#> 0.0002070461
#>
#> $statistic
#> z
#> 1.853763
#>
#> $p.value
#> [1] 0.06377312
#>
#> $parameter
#> n
#> 1000
#>
#> $conf.int
#> [1] -1.246385e-05 4.268371e-04
#> attr(,"conf.level")
#> [1] 0.95

# Get and plot Sen's slope of entire raster
test_loca_file <- system.file("extdata", "test_loca_sap_day.tif",
                              package = "mapler")
sap_prop <- terra::rast(test_loca_file)
sens_rast <- sens_slope_rast(sap_prop)
plot(sens_rast[[1]])
```



Future Climate Projection

The `sens` family of functions can be used for future climate projection. Output from `sap_day()` can be passed into `sens_farms()`, along with farms coordinates. This function combines the `get_sens_farms()` and `get_sens_joined()` functions. `sens_farms()` starts by calculating the Sen's slope for each farm and creating a new variable that is 1 if the slope is significant positive, -1 if the slope is significant negative, and 0 otherwise. Then, the farms are joined to a shapefile, and a proportion of the significant variable is calculated for each group in the shapefile. This resulting dataframe can be plotted with something like ggplot.

The following code can be used as a guide for plotting a future sap day projection.

```
library(rnaturalearth)
library(rnaturalearthdata)
library(sf)
library(tidyverse)
library(RColorBrewer)

# Read in farm coordinates and sap day projection (native package data)
farms_sf <- sf::st_as_sf(farms_coords, coords = c("lon", "lat"), crs = 4326)
```

```

test_loca_file <- system.file("extdata", "test_loca_sap_day.tif",
                             package = "mapler")
sap_prop <- terra::rast(test_loca_file)

# Read in eco regions shape file (native package data)
shapefile <- eco_regions
variable <- names(shapefile)[2]

# Get proportion of Sen's significance at each eco region
eco_regions_joined <- sens_farms(farms_coords = farms_sf, sap_prop = sap_prop,
                                shapefile = shapefile, group_var = variable)

# Prepare map for plotting (rnatrualearth functions)
world <- ne_countries(scale = "medium", returnclass = "sf")
north_america <- world |>
  filter(region_un == "Americas", name %in% c("United States of America", "Canada"))
us_states <- ne_states(country = "United States of America", returnclass = "sf")
canada_provinces <- ne_states(country = "Canada", returnclass = "sf")

# Create discrete breaks
all_levels <- levels(cut(seq(-1, 1, by = 0.2), seq(-1, 1, by = 0.2), include.lowest =
eco_regions_joined <- eco_regions_joined |>
  dplyr::mutate(cuts = factor(cut(sig_mean, seq(-1, 1, by = 0.2), include.lowest = TR
                                levels = all_levels))

# Add phantom rows for missing levels so the legend renders them
missing_levels <- all_levels[!all_levels %in% as.character(eco_regions_joined$cuts)]
dummy_rows <- eco_regions_joined[rep(1, length(missing_levels)), ] |>
  dplyr::mutate(
    cuts = factor(missing_levels, levels = all_levels),
    geometry = sf::st_sfc(lapply(seq_along(missing_levels),
                                function(x) sf::st_point(c(NA_real_, NA_real_))),
                        crs = sf::st_crs(eco_regions_joined))
  )
eco_regions_plot <- dplyr::bind_rows(eco_regions_joined, dummy_rows)

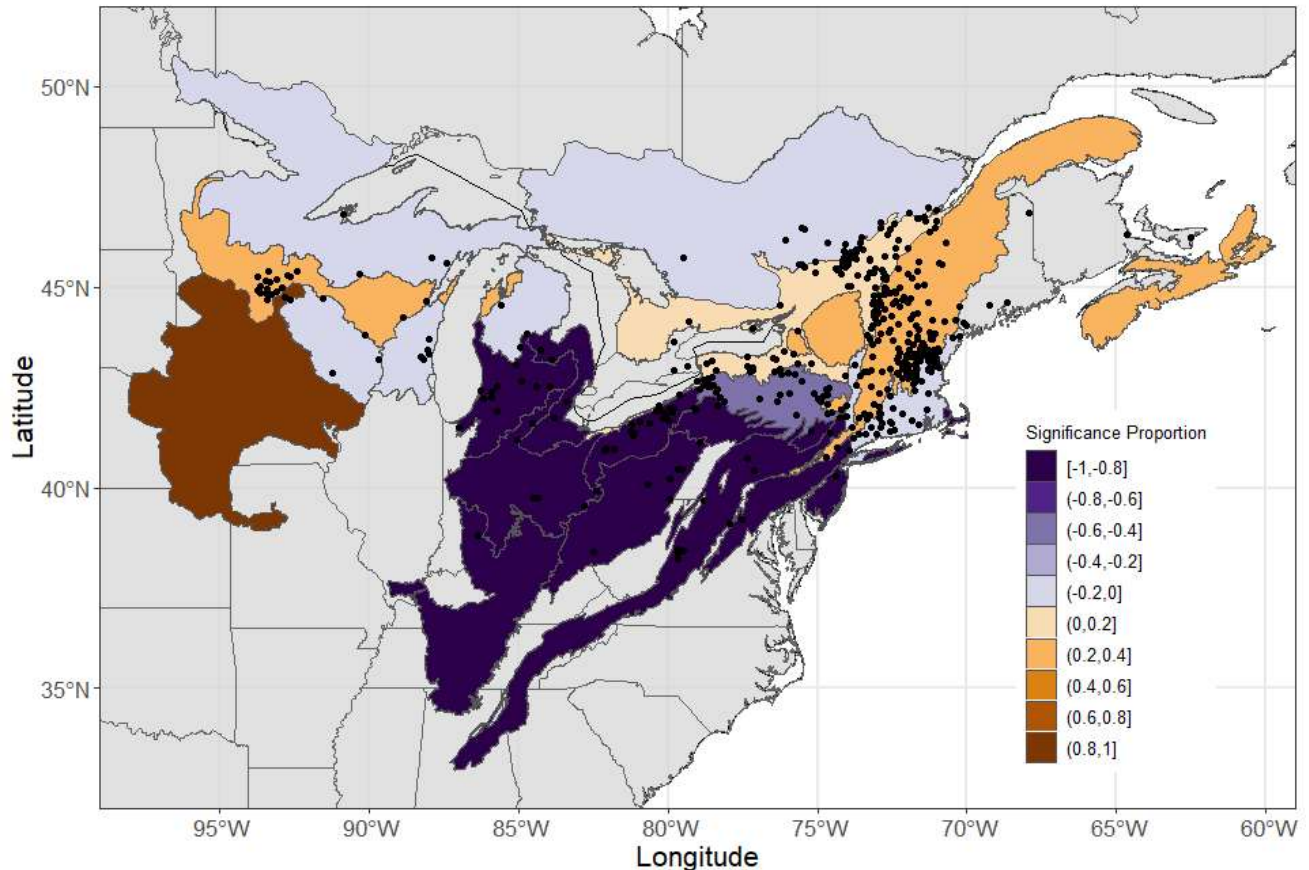
# Plot eco regions colored by significance proportion
ggplot() +
  geom_sf(data = north_america, fill = "grey85", color = "black", size = 0.2, alpha =
  geom_sf(data = us_states, fill = NA, color = "grey40", size = 0.3) +
  geom_sf(data = canada_provinces, fill = NA, color = "grey40", size = 0.3) +
  geom_sf(data = eco_regions_plot, mapping = aes(fill = cuts)) +
  geom_sf(data = farms_sf, color = "black", size = 1.5) +
  coord_sf(xlim = c(-99, -59), ylim = c(32, 52), expand = FALSE) +
  scale_fill_manual("Significance Proportion",
                    values = rev(brewer.pal(10, "PuOr")))) +
  theme_bw() +
  labs(
    x = "Longitude",

```

```

y = "Latitude"
) +
theme(legend.position = "inside",
      legend.position.inside = c(0.85, 0.27),
      text = element_text(size = 16),
      legend.text = element_text(size = 10),
      legend.title = element_text(size = 10),
      panel.grid.major = element_line(linewidth = 1))

```



The same thing can be done for U.S. counties:

```

library(tigris)

# Read in counties shapefile (tigris function)
shapefile <- counties(cb = TRUE, year = 2020)
#> |
variable <- "GEOID"

# Get proportion of Sen's significance at each U.S. county
us_counties_joined <- sens_farms(farms_coords = farms_sf, sap_prop = sap_prop,
                                shapefile = shapefile, group_var = variable)

```

```

# Create discrete breaks
all_levels <- levels(cut(seq(-1, 1, by = 0.2), seq(-1, 1, by = 0.2), include.lowest =
us_counties_joined <- us_counties_joined |>
  dplyr::mutate(cuts = factor(cut(sig_mean, seq(-1, 1, by = 0.2), include.lowest = TR
    levels = all_levels))

# Add phantom rows for missing levels so the legend renders them
missing_levels <- all_levels[!all_levels %in% as.character(us_counties_joined$cuts)]
dummy_rows <- us_counties_joined[rep(1, length(missing_levels)), ] |>
  dplyr::mutate(
    cuts = factor(missing_levels, levels = all_levels),
    geometry = sf::st_sfc(lapply(seq_along(missing_levels),
      function(x) sf::st_point(c(NA_real_, NA_real_))),
      crs = sf::st_crs(us_counties_joined))
  )
us_counties_plot <- dplyr::bind_rows(us_counties_joined, dummy_rows)

# Plot
ggplot() +
  geom_sf(data = north_america, fill = "grey85", color = "black", size = 0.2, alpha =
  geom_sf(data = us_states, fill = NA, color = "grey40", size = 0.3) +
  geom_sf(data = canada_provinces, fill = NA, color = "grey40", size = 0.3) +
  geom_sf(data = us_counties_plot, mapping = aes(fill = cuts)) +
  geom_sf(data = farms_sf, color = "black", size = 0.5) +
  coord_sf(xlim = c(-96, -59), ylim = c(34, 50), expand = FALSE) +
  scale_fill_manual("Significance Proportion",
    values = rev(brewer.pal(10, "PuOr"))) +
  theme_bw() +
  labs(
    x = "Longitude",
    y = "Latitude"
  ) +
  theme(legend.position = "inside",
    legend.position.inside = c(0.85, 0.27),
    text = element_text(size = 16),
    legend.text = element_text(size = 10),
    legend.title = element_text(size = 10),
    panel.grid.major = element_line(linewidth = 1))

```

